

1. Tempo disponibile 120 minuti.
2. Non è possibile consultare appunti, slide, libri, persone, siti web, ecc.
3. Scrivere in modo leggibile, su ogni foglio, nome, cognome e numero di matricola.
4. Le soluzioni agli esercizi che richiedono di progettare un algoritmo devono:
 - spiegare a parole l'algoritmo (se utile, anche con l'aiuto di esempi o disegni),
 - fornire e commentare lo pseudo-codice (indicando il significato delle variabili),
 - calcolare la complessità (con tutti i passaggi matematici necessari),
 - se l'esercizio ammette più soluzioni, a soluzioni computazionalmente più efficienti e/o concettualmente più semplici sono assegnati punteggi maggiori.

IMPORTANTE: Risolvere gli esercizi 1–2 e gli esercizi 3–4 su fogli separati. Infatti, al termine, dovreste consegnare gli esercizi 1–2 separatamente dagli esercizi 3–4.

1. Calcolare la complessità $T(n)$ del seguente algoritmo MYSTERY1

Algorithm 1: MYSTERY1(INT n) $\rightarrow$ INT

```

 $x = 0$ 
for  $i = 1, \dots, \log n$  do
   $x = x + \text{MYSTERY2}(i)$ 
return  $x$ 

function MYSTERY2(INT  $n$ )  $\rightarrow$  INT
if  $n \leq 0$  then
  return 1
else
   $k = n$ 
  for  $i = 1, \dots, n/2$  do
     $k = k - 1$ 
  return MYSTERY2( $k$ ) +  $k^2$ 

```

Soluzione. Analizziamo prima il costo di MYSTERY2. Tale funzione esegue un ciclo $n/2$ volte in cui ad ogni iterazione decrementa di 1 la variabile k , inizializzata ad n . Il ciclo ha un costo $\Theta(n)$ ed al termine di questo la variabile k ha valore $n/2$. Quindi la funzione MYSTERY2 esegue una chiamata ricorsiva con input $k = n/2$. L'equazione di ricorrenza che descrive il costo di MYSTERY2 è quindi

$$T'(n) = \begin{cases} 1 & n \leq 1 \\ T'(n/2) + n & n > 1 \end{cases}$$

Tale equazione di ricorrenza può essere risolta con il Master Theorem

$$\alpha = \log_2 1 = 0 < 1 = \beta \Rightarrow T'(n) = \Theta(n^\beta) = \Theta(n)$$

Il costo della funzione MYSTERY1 dipende dal costo del suo ciclo for interno. In tale ciclo viene richiamata la funzione MYSTERY2 $\log n$ volte per i che va da 1 ad $\log n$. Poiché il costo di MYSTERY2 è $\Theta(n)$ il costo complessivo di MYSTERY1 è dato da:

$$T(n) = \Theta(1) + \Theta(2) + \dots + \Theta(\log n) = \Theta\left(\sum_{i=1}^{\log n} i\right) = \Theta\left(\frac{\log n(\log n + 1)}{2}\right) = \Theta(\log^2 n)$$

2. Sia L una lista doppiamente concatenata in cui ogni nodo, oltre ai puntatori *next* e *prev*, contiene un campo *char* per memorizzare una carattere. Tale lista può essere utilizzata per memorizzare una stringa (un solo carattere per nodo).
 - a) Scrivere lo pseudocodice di un algoritmo che, presa la lista L in input, verifichi se la stringa rappresentata è palindroma. Esempio:
 - "radar" è una stringa palindroma
 - "antenna" non è una stringa palindroma
 - b) Analizzare il costo ottimo e pessimo della soluzione proposta

Nota. Una stringa vuota è considerata palindroma.

Soluzione.

- a) Dato che la lista L è doppiamente concatenata, possiamo risolvere il problema implementando un algoritmo ricorsivo *ausiliario* che prende in input un puntatore ad un nodo in testa ed uno ad un nodo in coda. Se i caratteri nei nodi puntati sono uguali il puntatore in testa viene aggiornato in modo da puntare al nodo successivo, e quello in coda al nodo precedente. E' necessario prestare attenzione alle condizioni di terminazione poiché la ricorsione termina quando:
 - i caratteri puntati dai due puntatori sono differenti, oppure
 - entrambi i puntatori puntano allo stesso nodo, oppure
 - puntano a due nodi adiacenti

Una funzione di interfaccia si occupa di individuare il primo ed ultimo nodo e di lanciare la funzione ausiliaria.

Algorithm 2: ISPALINDROME(LIST L) $\rightarrow$ BOOL

```

if  $L == \text{NIL}$  then
  | return true
else
  |  $head = tail = L.head$ 
  | while  $tail.next \neq \text{NIL}$  do
  |   |  $tail = tail.next$ 
  |   | return ISPALINDROME(AUX( $head, tail$ ))

function ISPALINDROME(AUX(NODE  $head$ , NODE  $tail$ )  $\rightarrow$  bool
if  $head.char \neq tail.char$  then
  | return false
else if  $head == tail$  or  $head.next == tail$  then
  | return true
else
  | return ISPALINDROME(AUX( $head.next, tail.prev$ )

```

- b) Valutiamo prima il costo ottimo e pessimo di ISPALINDROME(AUX)
 - costo ottimo: $O(1)$ (il primo e ultimo carattere sono differenti)
 - costo pessimo: $\Theta(n)$ (la stringa è palindroma, dobbiamo visitare tutta la lista)

D'altra parte, indipendentemente dal fatto che la stringa rappresentata sia palindroma la funzione ISPALINDROME deve visitare interamente almeno una volta la lista per individuare l'ultimo nodo (ciclo while). Di conseguenza, il costo ottimo e pessimo di ISPALINDROME è $\Theta(n)$.

3. Progettare un algoritmo che utilizza la programmazione dinamica per risolvere il seguente problema: dato un array $A[1..n]$ di numeri interi positivi ed un numero intero $K \geq 0$, restituire il numero minimo di valori presenti in A che sommati danno K . Se nell'array A non esistono valori che sommati danno K , l'algoritmo deve restituire ∞ .

Soluzione. È possibile utilizzare la programmazione dinamica considerando i problemi $T(i, j)$, con $i \in \{1, \dots, n\}$ e $j \in \{0, \dots, K\}$, tali che:

$T(i, j)$ = numero minimo di elementi del sotto-array $A[1..i]$ che sommati danno j . Nel caso in cui non ci siano numeri in $A[1..i]$ che sommati danno j , poniamo $T(i, j) = \infty$.

Il problema iniziale coincide con $T(n, K)$.

I problemi $T(i, j)$ possono essere risolti considerando che:

$$T(i, j) = \begin{cases} 0 & \text{se } i = 1 \text{ e } j = 0 \\ \infty & \text{se } i = 1 \text{ e } j > 0 \text{ e } A[1] \neq j \\ 1 & \text{se } i = 1 \text{ e } j > 0 \text{ e } A[1] = j \\ T(i-1, j) & \text{se } i > 1 \text{ e } j < A[i] \\ \min\{T(i-1, j), 1 + T(i-1, j - A[i])\} & \text{se } i > 1 \text{ e } j \geq A[i] \end{cases}$$

Algorithm 3: MINIMACARDINALITASOMMATORIA(INT $A[1..n]$, INT K) $\rightarrow$ INT

```

INT T  $\leftarrow$  new INT[1..n, 0..K]
// Inizializzazione prima riga della tabella T
for  $j \leftarrow 0$  to  $K$  do
    if  $j = 0$  then
        |  $T[1, j] \leftarrow 0$ 
    if ( $j > 0$  and  $A[1] \neq j$ ) then
        |  $T[1, j] \leftarrow \infty$ 
    else
        |  $T[1, j] \leftarrow 1$ 
// Riempimento restanti righe delle tabella T
for  $i \leftarrow 2$  to  $n$  do
    for  $j \leftarrow 0$  to  $K$  do
        if  $j < A[i]$  then
            |  $T[i, j] \leftarrow T[i-1, j]$  // non si può utilizzare l' $i$ -esimo numero
        else if ( $T[i-1, j] < 1 + T[i-1, j - A[i]]$ ) then
            |  $T[i, j] \leftarrow T[i-1, j]$  // non conviene utilizzare l' $i$ -esimo numero
        else
            |  $T[i, j] \leftarrow 1 + T[i-1, j - A[i]]$  // conviene utilizzare l' $i$ -esimo numero
// Restituzione della soluzione al problema iniziale
return  $T[n, K]$ 

```

L'Algoritmo 3 risolve i problemi $T(i, j)$ inserendo le relative soluzioni nella tabella $T[1..n, 0..K]$. Al termine dello riempimento della tabella, si restituisce il valore in $T[n, K]$. Il costo computazionale di tale algoritmo risulta essere $\Theta(n \times K)$ in quanto le istruzioni hanno tutte costo costante, ed il blocco di maggiore costo è quello che contiene i due **for** annidati le cui istruzioni interne vengono eseguite $(n-1) \times (K+1)$ volte.

4. Progettare un algoritmo che dato un grafo orientato $G = (V, E)$, un vertice $v \in V$ ed un numero K , restituisce *true* se esiste un ciclo semplice di lunghezza K che include v , *false* altrimenti.

Soluzione. Si può procedere cercando tutti i cammini semplici (ovvero privi di cicli) di lunghezza $K-1$ che iniziano da v e verificare se almeno uno di questi cammini raggiunge un vertice che ha v tra le proprie adiacenze. Questi cammini possono essere cercati con una versione modificata dell'algoritmo DFS che procede fino a profondità $K-1$ e che, al momento della chiusura della visita di un vertice, toglie la marcatura perché quel vertice potrebbe essere rivisitato in quanto appartiene ad altri cammini che vengono analizzati successivamente. L'algoritmo termina appena si trova un cammino con le caratteristiche sopra riportate.

Tale idea è implementata dall'Algoritmo 4. Si noti che la funzione che implementa la versione modificata della DFS utilizza un parametro aggiuntivo l che indica la lunghezza residua del cammino da trovare. Dal punto di vista del costo computazionale, nel caso in cui non ci sia il ciclo l'Algoritmo deve provare tutti i possibili cammini semplici di lunghezza $K-1$ che partono da v in un grafo con $n = |V|$ vertici. Nel caso pessimo il numero di tali cammini è $(n-1) \times (n-2) \times \dots \times (n-K+1)$, in

quanto ogni nuovo vertice del cammino deve essere differente da quelli già attraversati. Abbiamo quindi che il costo computazionale di tale algoritmo è $O(n^k)$.

Algorithm 4: CONTROLLACICLO(GRAPH (E, V) , VERTEX v , INT K) $\rightarrow$ BOOLEAN

```
for  $v \in V$  do
   $v.mark \leftarrow false$  // All'inizio tutti i vertici non sono marcati
return DFSMODIFIED( $v, K - 1$ ) // Cerca un cammino da  $v$  di lung. $K - 1$  da cui si torna a  $v$ 

// Cerca un cammino da  $u$  di lunghezza  $l$  che arriva a un vertice dal quale si raggiunge  $v$ 
DFSMODIFIED (VERTEX  $u$ , INT  $l$ )  $\rightarrow$  BOOLEAN
if  $l == 0$  then
  return ( $v \in u.adjacents()$ )
else
   $u.mark \leftarrow true$ 
  for  $w \in u.adjacents()$  do
    if not  $w.mark$  then
      if DFSMODIFIED( $w, l - 1$ ) then
        return true
   $u.mark \leftarrow false$  // Si toglie la marcatura del vertice per successive ricerche
  return false
```
